Earning Roles: Situated Peer Evaluation and Dynamic Responsibility Assignment Inspired by Human Social Collaboration

Anonymous Author(s)
Submission Id: Anonymous submission

Abstract

In human collaboration, expertise is rarely established by self-declaration alone. It emerges as partners use one another's work, evaluate concrete deliveries, and decide whom to entrust with similar responsibilities next. Roles fixed before collaboration lack this adjustment mechanism. We translate this principle into *earning roles*: an agent's role is its task-situated eligibility for future responsibility, not a permanent title or a context-free reputation score. The system records a recipient's acceptance, revision, or rejection of a versioned artifact together with its use context as typed, noisy, auditable experience. Peer evaluation is exposed to subsequent selection as an observation, while producer responsibility is decided by an independent attribution check. Selector parameters change only after a later assignment has been executed and its independent quality, responsibility, and complete cost have passed validation; valid feedback may record either success or failure, whereas missing or unattributable outcomes do not update the policy. We evaluate this complete chain in an ArtifactRole track and isolate local candidate sets, delayed feedback, and online service behavior in a complementary PeerSelect track. All comparisons use matched candidates, public information, budgets, and feedback timing. RARE attains a future quality–cost utility of xx, differing from the same-information primary baseline by xx (stream-level 95% CI: xx); judgment Brier score, false producer-attribution rate, and update p95 are xx, xx, and xx. Ablations separate the effects of peer information, responsibility validation, and delayed updating.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; *Cooperation and coordination.*

Keywords

multi-agent systems, human social collaboration, dynamic roles, peer evaluation, online adaptation, responsibility attribution

ACM Reference Format:
Anonymous Author(s). 2027. Earning Roles: Situated Peer Evaluation and Dynamic Responsibility Assignment Inspired by Human Social Collaboration. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), Hanoi, Vietnam, 3–7 May 2027*, IFAAMAS, 11 pages.



This work is licensed under a Creative Commons Attribution International 4.0 License.

Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), M. Baldoni, F. Fang, W. Yeoh, N. Yorke-Smith (eds.), 3–7 May 2027, Hanoi, Vietnam. © 2027 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org).

1 Introduction

In human societies, professional standing is seldom determined by self-description. It develops when collaborators use a person's work, evaluate whether the delivery was useful or required repair, and decide whether to entrust related responsibilities to that person again. Research on transactive memory describes how teams develop shared knowledge of who knows what through collaboration [19, 33]. We adopt this rule without treating social reputation as ground-truth ability: value is inferred from concrete work and tested by subsequent delegation.

Role prompts and fixed handoff orders make multi-agent workflows practical [14, 36], but a role name states what an agent is expected to do, not what it has demonstrated in collaboration. Coordination depends on managing dependencies between activities [24]. When tasks or member performance change, a workflow that ignores collaboration experience continues to reuse its original allocation. We therefore define *earning roles* as acquiring eligibility for future responsibility in a task context. A role is not a permanent position; expertise is not self-declared; and earning does not confer ownership forever.

Our evaluator is the agent that actually receives and uses a versioned delivery. Its acceptance, revision, or rejection is stored with the task, artifact, and use trace as a *collaboration experience*. A selector reads these experiences on a later task. An independent responsibility check determines which outcomes are attributable to the producer, and the selected agent's subsequent quality and complete cost determine whether that allocation should be repeated. Peer evaluation, responsibility attribution, and future assignment thus retain distinct meanings.

Consider a queue-processing workflow. Agent B supplies a priority module and agent A integrates it into a consumer. When another task requires queue processing, the selector may use A's account of whether and how the module worked, rather than rely only on B's self-description. Yet a rejection is not automatically evidence of a producer defect. The same failed team outcome can arise because B violated the module contract or because B's module was correct while A's consumer was faulty. The system must retain the recipient judgment, actual use, and independent quality check to distinguish these cases.

We instantiate this principle as the earning-roles protocol. The selector combines current requirements, candidate information, and prior experiences before execution. Positive and negative recipient judgments remain observations; producer responsibility is validated separately. Publishing an observation can change a later selector input but cannot directly update its parameters. Only a later, executed assignment with independently validated quality

and complete cost can update the selector. A judgment is therefore not converted into immediate reward or punishment.

We call the responsibility-qualified component *responsibility-aware role evidence* (RARE) and store it separately from raw evaluation. Figure 1 shows how this distinction supports subsequent delegation. The central test compares RARE with acceptance-only, terminal-only, and same-information contextual selectors to measure its effect on unseen-task collaboration quality and its evaluation, validation, and update costs.

The paper makes three contributions.

- We define task-situated collaboration experience that records who evaluated whose versioned delivery, how it was used, and which outcomes are attributable to the producer. Both positive and negative evaluations remain auditable in context.
- We separate observation publication from delayed policy updating. Published evaluations inform later choices, whereas parameters update only from independent outcomes of executed assignments; factorial ablations isolate the two paths.
- We introduce an ArtifactRole track for the complete delivery–evaluation–delegation–utility chain and a PeerSelect track for local selection, delay, drift, and service behavior. Same-information baselines and responsibility interventions separate information value, attribution error, and gains from additional information.

2 Related Work and Positioning

Peer feedback has recently been used to adapt multi-agent collaboration. Meta-Team collects post-task accounts of how teammates affect downstream execution and updates teammate profiles and collaboration arrangements [12]. RAPS maintains peer reputation from first-hand content evaluations and witness reports [22]. Sero studies contract-preserving role evolution [9], C3 studies exact credit assignment among cooperative agents [6], and DecisionBench delegates from externally judged capability cards built from prior trajectories [8]. These works cover collaborative reflection, witness propagation, contract-bound roles, credit assignment, and historical capability summaries, respectively. They do not jointly bind a recipient's use of one versioned delivery, a producer-responsibility gate, and a later executed responsibility opportunity into one auditable chain. Our analysis separates the evaluation of a concrete delivery, producer attribution, and the utility of subsequent delegation.

Computational trust and reputation provide the broader foundation for using interaction history in open multi-agent systems [26, 30]. Trust models also account for inaccurate reports by third parties [31]. Partner-selection methods adapt from social-game payoffs [29, 34, 35], while language-agent societies use shared assessments or repeated interaction to sustain cooperation [2, 25, 28]. In the cited formulations, interaction-level feedback does not explicitly represent the producer–recipient artifact-ownership fields needed here.

Graph and role systems reorganize agents from traces, routing outcomes, contexts, or subtask performance [21, 37, 40, 42]. General multi-agent frameworks likewise make role-based workflows practical [5, 14, 36]. Our object is narrower: an action on a specific delivery is separated from producer responsibility and from the payoff of a later assignment. Table 1 summarizes this boundary.

Task-allocation taxonomies distinguish who performs which task under different organizational constraints [10]. Cooperative multi-agent learning separates shared return from individual contribution through counterfactual credit or value factorization [7, 27]. These are related credit questions; our distinction concerns a versioned delivery used inside a recipient's integration work. Contextual bandits and delayed-feedback learning provide strong update controls [15, 20]. Here, source evaluation is visible before a later choice, whereas target quality exists only after execution and cannot be backfilled into the earlier decision.

3 Problem Formulation

3.1 Tasks, assignments, and collaboration experience

At task t, context x_t and legal candidate set C_t specify requirements and delivery responsibility. Before execution, the selector chooses

$$a_t \sim \pi_{\psi,\Theta_t}(\cdot \mid x_t, C_t, R_t), \qquad p_t = \pi_{\psi,\Theta_t}(a_t \mid x_t, C_t, R_t), \tag{1}$$

where $a_t \in C_t$, $p_t \in (0, 1]$, R_t is the public experience available at that time, ψ denotes fixed scoring parameters, and Θ_t is online state. Public records and parameters are maintained separately, so a new evaluation can change the choice input without immediately changing parameters.

A source delivery forms the record

$$e_i^{\mathrm{src}} = (x_i, C_i, a_i, o_i, j_i, J_i, m_i, Q_i, Y_i^{\mathrm{src}}, \tau_i, c_i^{\mathrm{src}}). \tag{2}$$

Here o_i is producer a_i's versioned artifact; j_i is the recipient that uses it; J_i is sealed before the recipient acts; m_i is the subsequent use, revision, or rejection; Q_i checks the original delivery against the producer's responsibility; Y_i^{src} is the source episode's independently defined terminal or later-use outcome, or $\perp$ if the contract defines none; τ_i is publication order; and c_i^{src} records model, tool, token, and wall-clock cost. Source outcomes support observation and audit, not future-assignment credit.

For a target assignment j, the independent outcome is Y_j^{tar}. After the assignment is sealed, its candidates and probability are consistent, and execution completes, Y_j^{tar} enters validate_later with that assignment to produce feedback L_j. Only an L_j that passes responsibility, quality, complete-cost, and idempotency checks updates Θ. Hence Y_i^{src} and Y_j^{tar} are distinct, and neither is retroactively available to an earlier choice.

For example, let the queue task offer $C_t = \{B, C\}$. The selector assigns the source delivery to B, whose priority module is received by A. Then Q_i checks the module for which B is responsible, m_i records how A integrates or changes the code, and Y_i^{src} records only the terminal or later-use result defined for that source episode. If A repairs only its own consumer, the experience can be retained, but that repair is not evidence of a defect in B's module. Only a subsequent target outcome that produces valid L_j can affect future parameters. This example also explains why an episode identifier alone is insufficient: the source artifact, recipient action, and later assignment serve different evidential roles.

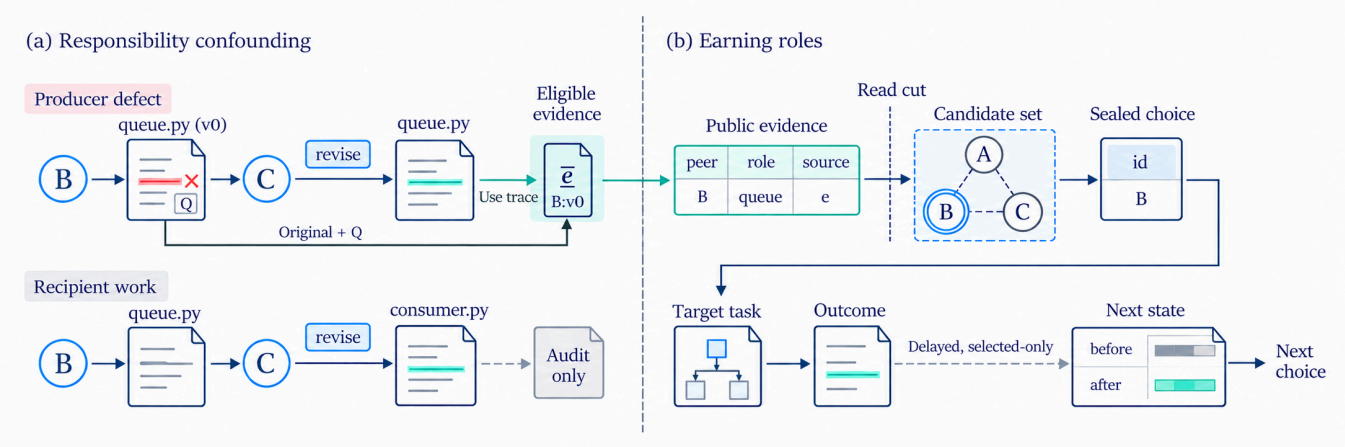


Figure 1: From peer evaluation to future responsibility. (a) Both cases receive a revision judgment, but responsibility differs. An independent check Q of producer B's original version $v0$, together with the recipient's use trace, determines whether the observation forms evidence e attributable to B. (b) Collaboration experiences are read before a later assignment; the executed assignment changes subsequent selection only after outcome validation. File changes, candidate state, and selections are schematic.

Method family	Observation and update	Responsibility / credit	Experimental object	Difference tested here
Meta-Team; RAPS [12, 22]	Post-task accounts or first-/second-hand evaluation; profiles or reputation update	Teammate or content usefulness	Adaptive collaboration / publishing	Versioned use, independent producer gate, later assignment utility
Sero; C3; DecisionBench [6, 8, 9]	Contract evolution, precise team credit, or judged trajectory cards	Role constraints, contribution credit, or capability summaries	Reasoning, cooperation, or delegation	Recipient action and producer attribution remain separate
Partner selection / reputation [28, 29, 35]	Interaction outcomes and shared assessments	Peer payoff or pooled reputation	Social games / language-agent society	Producer–recipient artifact ownership
AgentNet; GPTSwarm [37, 42]	Traces, graph structure, and routing outcomes	Route or role utility	Graph / workflow coordination	Action bound to one delivery and future owner
Dynamic roles / experts [21, 40]	Context and role/subtask performance	Role or expert utility	Debate / mixture-of-experts workflow	Responsibility gate and delayed correction
Finite-time bandit [3]	Selected-arm reward after execution	Arm-level reward	Pre-execution choice	Context, ownership, and delay require qualification

Table 1: Comparison dimensions. Adapted experiments preserve matched public information, feedback timing, and complete cost; rows denote different research objects rather than interchangeable implementations.

3.2 Visible information and objective

The selector reads a public projection $\tilde{e}_i = \Pi_{\text{pub}}(e_i)$ rather than the audit record. The projection preserves agreed task fields, candidate identity, recipient judgment, actual action, and responsibility type; hidden answers, private tests, and future outcomes remain unavailable. Given context x, candidate a, and public history R_t, situated assignment value is

$$V_t(a \mid x, R_t) = \mathbb{E}[U(Y_j^{\text{tar}}, c_j^{\text{tar}}) \mid x, R_t, a], \tag{3}$$

where U is the frozen quality–complete-cost utility. This quantity asks where a candidate should again receive responsibility, not whether it has context-free global ability. Producer quality, recipient evaluation, and team outcome are recorded separately. Team success does not prove that every member performed well; a negative judgment does not automatically label the producer. Self-description supplies at most an initial prior and never verified evidence.

4 Earning Roles

The protocol preserves evaluations from actual collaborators, organizes task-relevant history, assigns the next responsibility, and calibrates the selector from the executed result. Responsibility validation distinguishes what a peer observed from what can be attributed to the producer. The auditable order is

$$\begin{aligned} &\text{delivery} \rightarrow \text{sealed judgment} \rightarrow \text{use} \\ &\quad \rightarrow \text{publication} \rightarrow \text{assignment} \\ &\quad \rightarrow \text{execution} \rightarrow \text{validated update}. \end{aligned} \tag{4}$$

Event	Sealed information and state transition
1. Deliver	Record producer, responsibility contract, artifact version o_i, and source cost.
2. Judge	The recipient seals J_i before using or editing the delivery; no policy update occurs.
3. Use	Record acceptance, revision, or rejection as m_i, together with the actual use context.
4. Publish	Bind the observation and independent check in e_i^{src}; publish by arrival order, leaving Θ unchanged.
5. Assign	At w_t, resolve legal history and seal the candidate menu, input snapshot, chosen a_t, and propensity p_t.
6. Execute	Run the sealed choice without resampling; obtain Y_j^{tar} and complete cost for the selected assignment.
7. Validate	Match feedback to its assignment. Only valid, selected-only, once-only L_j updates later residual state.

Table 2: Operational order across a source delivery and a subsequent target assignment. A source evaluation is never the outcome of an unexecuted target.

A later result cannot change an earlier assignment.

Table 2 specifies what is recorded at each boundary. Source evaluation and target execution can occur in different tasks: the source contributes a collaboration experience, while the target supplies feedback about an actual allocation. Publishing the former does not imply that the latter has already happened.

4.1 Observation and responsibility

We define observation completeness $O_i \in \{0,1\}$ and producer-attribution eligibility $G_i \in \{0,1\}$. $O_i = 1$ when the evaluation, versioned delivery, recipient action, and independent check are completely bound under the prescribed time order. It does not depend on whether the recipient accepts, revises, or rejects; all three remain collaboration experience.

$G_i = 1$ adds responsibility requirements: the original delivery maps uniquely to a predeclared producer duty, the recipient actually used it, the independent check supports attribution, and the evidence does not mix recipient-owned integration changes. Responsibility follows the task contract, artifact lineage, and actual modifications rather than the evaluator's assertion. Recipient-only work, mixed ownership, or unresolved responsibility does not generate producer reward or penalty. Records with $O_i = 1, G_i = 0$ remain typed contextual observations but cannot be reinterpreted as producer-quality labels. In the queue example, A's integration difficulty may affect future collaboration choice, but it cannot become evidence that B's module is defective.

4.2 History and assignment

At read cut w_t, candidate version v exposes

$$\mathcal{E}_t^{\text{obs}}(v) = \text{Resolve}_{w_t}\{\tilde{e}_i : O_i = 1, a_i = v, \tau_i \le w_t\}, \tag{5}$$

$$\mathcal{E}_t^{\text{cred}}(v) = \text{Resolve}_{w_t}\{\tilde{e}_i : O_i = 1, G_i = 1, a_i = v, \tau_i \le w_t\}. \tag{6}$$

Resolve_{w_t} deduplicates by source and retains only the valid version known at w_t. Different versions of one agent remain distinct candidates. Records are filtered by publication arrival, not task occurrence, so late evaluations affect only later choices. The public projection retains judgment category, recipient identity, use context, and attribution type.

Globally, $\mathcal{R}_t^{\text{obs}}$ retains complete recipient observations, including records with unresolved responsibility, whereas $\mathcal{R}_t^{\text{cred}}$ contains attributable evidence. Publishing either view does not change Θ, the update count, or the policy digest. Records with $G_i = 0$ may enter context as unknown or mixed responsibility, but cannot generate source credit or trigger a parameter update by themselves. Only a validated later L_j can trigger updating. The gate therefore changes the meaning and update eligibility of evidence without silently discarding observations that lack attribution.

Explicitly, the two public views at a common read cut are

$$\mathcal{R}_t^{\text{obs}} = \{\tilde{e}_i : O_i = 1, \tau_i \le w_t\}, \tag{7}$$

$$\mathcal{R}_t^{\text{cred}} = \{\tilde{e}_i : O_i = 1, G_i = 1, \tau_i \le w_t\}. \tag{8}$$

Thus $\mathcal{R}_t^{\text{cred}} \subseteq \mathcal{R}_t^{\text{obs}}$: attribution qualifies a subset without turning the remaining observations into failed deliveries. Attribution coverage and observation coverage can therefore change independently, and both are reported.

History is divided into an archive and a new delta:

$$P_{v,k} = \text{summarize}\{\tilde{e}_i \in \mathcal{E}_t^{\text{obs}}(v) : \tau_i < \kappa_k\}, \tag{9}$$

$$D_{v,k,t} = \text{summarize}\{\tilde{e}_i \in \mathcal{E}_t^{\text{obs}}(v) : \kappa_k \le \tau_i \le w_t\}, \tag{10}$$

$$z_t(v) = [\phi(x_t) \,\|\, \phi(P_{v,k}) \,\|\, \phi(D_{v,k,t}) \,\|\, b_t(v)]. \tag{11}$$

The structured summaries retain source identifiers; empty history is explicit. The archive cut satisfies $\kappa_k \le w_t$, and archive and delta never duplicate a source in the same snapshot. Corrections rebuild an affected archive before its next read, advance the archive version, and repartition the delta. This prevents an obsolete record from remaining in the archive while its correction is counted again in the delta.

The frozen encoder $\phi(\cdot) \in \mathbb{R}^d$ represents the task, archive, and delta. Public version and coverage fields form $b_t(v) \in \mathbb{R}^{d_b}$, so concatenation yields $z_t(v) \in \mathbb{R}^{3d+d_b}$. The archive summarizes others' evaluations of actual work; self-description supplies only an initial reference. Joint task and history encoding allows the same candidate to receive different responsibility opportunities under different requirements.

A fixed head and candidate-specific residual define

$$s_t(v) = g_\psi(z_t(v)) + \theta_{v,t}^\top \phi(x_t). \tag{12}$$

Here $g_\psi : \mathbb{R}^{3d+d_b} \to \mathbb{R}$ is fixed during online operation, while $\theta_{v,t} \in \mathbb{R}^d$ is the candidate's residual and $\Theta_t = \{\theta_{v,t}\}_v$. The first term reads current collaboration experience; the second calibrates scores from previously validated execution outcomes. With temperature $T > 0$ and exploration $0 < \varepsilon < 1$,

$$q_t(v) = \frac{\exp(s_t(v)/T)}{\sum_{u \in C_t} \exp(s_t(u)/T)}, \tag{13}$$

$$\pi_t(v) = (1-\varepsilon) q_t(v) + \varepsilon/|C_t|, \qquad a_t \sim \pi_t. \tag{14}$$

Every candidate therefore retains nonzero opportunity. The assignment record seals the candidate set, sample, probability, and history version before target execution. Unselected candidates receive no outcome labels. Complete cost includes selected-agent work, recipient integration, evaluation, and validation.

State	Information and time of change
Public record R	Versioned, typed experience; changes at publication or correction arrival.
Archive $P_{v,k}$	Earlier valid experience; refreshed at archive cuts or corrected-source rebuilds.
Delta $D_{v,k,t}$	New valid experience after the archive cut; read at the next selection.
Residual Θ	Task-related score correction; changes once per validated assignment feedback.
Assignment a_t, p_t	Selected candidate and propensity; fixed before execution, never overwritten.

Table 3: Experience and parameters change at different times.

More precisely, $\pi_t(v) \geq \varepsilon/|C_t|$ for every legal candidate. This floor leaves an opportunity for members with little recorded experience; it does not assign them a success or failure label in advance. The temperature and exploration rate remain fixed by the experimental configuration. The selected identity and its probability describe the choice actually executed, rather than a new sample drawn after the outcome is known.

4.3 Delayed updating

Publication and parameter updating are independent:

$$\text{publish}(\tilde{e}_i) : R \leftarrow R \cup \{\tilde{e}_i\}, \tag{15}$$

$$\text{update}(L_j) : \Theta^+ = U_{\text{delay}}(\Theta; L_j). \tag{16}$$

Publication leaves Θ, the update count, and the policy digest unchanged. L_j binds the pre-execution assignment, propensity, and input snapshot to the independent target outcome, responsibility check, quality, and complete cost. The frozen activation manifest specifies representation, updater, initialization, clipping, batching, and stopping constants. Validation checks assignment order, candidate identity, outcome completeness, reward semantics, and once-only use. Valid successes and failures both form feedback; missing or unattributable labels do not update.

Each update uses the selected assignment's sealed input, rather than reconstructing that decision with a newer archive. It calibrates future assignment value; it does not relabel the source delivery. The source recipient's own integration work cannot directly reward or penalize the producer, and a target team's terminal result alone is insufficient as a producer-ability label. Duplicate feedback is counted once. Corrections and delayed records affect only choices after their arrival. Table 3 makes the different update times explicit.

This separation lets new evaluations inform a decision while parameters remain fixed, and lets subsequent execution correct longer-term allocation. Experiments disable evaluation reading and parameter updating separately. Responsibility interventions test whether recipient-owned work is mistakenly converted into producer reward or punishment.

5 Benchmarks and Baselines

ArtifactRole. The primary track uses TeamBench's dependency structure [17] to expose producer delivery, recipient use and evaluation, responsibility checks, later assignment, and complete cost. DIST1 queue-race and PIPE3 stream-processing separate development and confirmation roots; changing a random seed does not create a new structural root. A root identifies a task-generation mechanism and dependency structure. Results are reported per root to distinguish adaptation within a task family from transfer across tasks. Producer contract conformance and terminal usability are scored separately because a recipient may normalize a nonconforming delivery into a usable result. Every executed arm, including no-update, receives terminal evaluation. Native benchmark conformance remains a separate metric. Valid assignment credit is required for updating, not for entry into the intention-to-treat outcome denominator.

PeerSelect. The secondary track uses a versioned benchmark of interactions on a local graph to isolate selected-only payoff, local candidate sets, online updating, drift, and service cost. It cannot establish artifact-specific judgment or producer attribution. The two tracks share event and selector interfaces while retaining separate tasks, labels, scorers, denominators, and claims.

Agent benchmarks supply complementary coordination and interaction coverage [1, 23, 32, 38, 41]. They do not, by default, expose the proposed producer–recipient lineage. All primary arms receive matched candidate menus, initial opportunities, model/API/tool budgets, exploration, feedback arrival schedules, failure handling, and complete-cost accounting.

RLS, online logistic regression, SGD, and periodic refitting are compared under the same selection interface. This separates updater computation from responsibility-aware information use. Continual-learning and multi-agent non-stationarity motivate the drift and forgetting analyses [13, 16]; agent ranking and human–LLM teaming motivate generality and deployment boundaries [18, 39].

6 Experimental Questions and Matrix

RQ1 (information value). Does eligible situated evaluation predict an independent producer-contract or later-use result beyond raw acceptance and terminal-only feedback? Measures are held-out log loss, Brier score [4], AUC when defined, calibration [11], and incremental value on an independent root.

RQ2 (assignment causality). Does public evidence change an assignment sealed before target execution, including transfer to a future owner different from the source producer? Measures are assignment-change probability, propensity-weighted future quality, adoption, rework, and complete cost. Evidence about B cannot be consumed as evidence about C.

RQ3 (online behavior). Does delayed selected-only updating satisfy the service contract? Measures are publication, selection, and update p50/p95 latency; service lag; backlog; state bytes; processor, accelerator, and memory use; tokens, tools, and API calls; and replay time.

RQ4 (stability and boundaries). How does the method behave under task or peer drift, delayed or out-of-order feedback, version replacement, recipient-only work, mixed ownership, scorer failure, and resource failure? Measures are peak and mean old-root forgetting, recovery window, UNKNOWN rate, correction consistency, and false producer attribution.

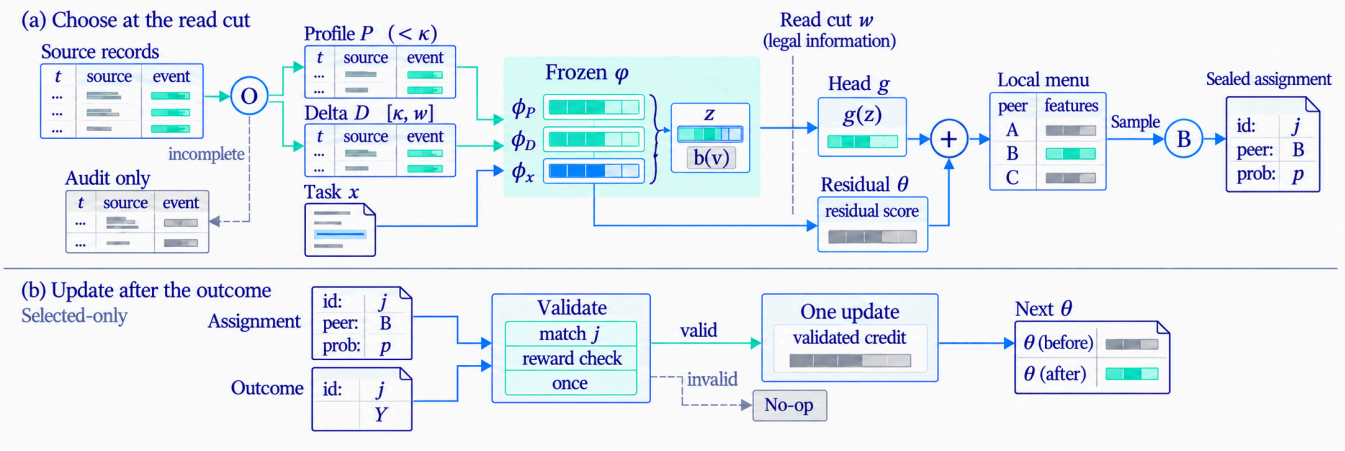


Figure 2: Experience reading and delayed updating. (a) Effective history is split into archive P and delta D, then combined with task representation and public metadata. The candidate and propensity are sealed before execution. (b) After execution, the matching independent quality and cost outcome must pass identity, responsibility, and once-only checks before updating the residual used by later decisions.

Arm	Read / update contract	Alternative explanation tested
Uniform	Candidate menu; fixed local-uniform sampling	Random allocation and exploration floor
No update	Initial prior and current context; feedback ignored	Static execution is sufficient
Raw acceptance	Recipient acceptance / rejection only	Responsibility filtering adds no value
Terminal only	Independent terminal outcome only	Situated evaluation adds no information
Contextual trust / bandit	Same context, evaluation, propensity, delay, and budget as RARE	Ordinary contextual learning explains the effect
Pooled controller	Permitted public history at matched total capacity	Shared control explains the gain
Closest published adapter	Public-information adapter preserving source semantics	Existing role / organization method is sufficient
RARE	Observation and responsibility separated; validated selected assignment updates	Complete evaluation–delegation–calibration mechanism

Table 4: Predeclared baselines. Same-information comparisons preserve context, feedback timing, and complete budget.

The uncertainty unit is an independently reset stream; events within a stream are dependent. We use paired stream-level estimates, bootstrap or randomization intervals over streams, and preregistered multiplicity control. Missing, invalid, and unknown events remain in their declared denominators. H1 is judgment Brier score, H2 future quality–complete-cost utility, H3 update p95 and backlog, and H4 false producer-attribution rate. H2 is higher-is-better; the others are lower-is-better. Comparison direction, precision target, 95% interval, and stopping rule are sealed before confirmation outcomes are read.

Denominators. Eligibility for learning must not filter the population used to estimate team performance. Table 6 distinguishes the six relevant counts. Always $N_{\text{credit}} \leq N_{\text{outcome}}$; credit eligibility never filters the H2 intention-to-treat denominator. Predictive metrics use only their predefined scorable-label subset while reporting the full denominator and reasons for missingness. Unselected candidates have no outcome labels and are not treated as failed samples.

The count hierarchy locates failures without conflating them with learning eligibility. A scheduled target that never starts contributes to planned load and stopping-rule accounting, but not to the outcome denominator. Once a target starts, a scorer failure, invalid check, unknown responsibility, or resource failure remains in the H2 population together with its incurred cost. That case may lack feedback eligible for updating, so update coverage is reported separately from team utility. Prediction scores use their predefined scorable labels, accompanied by the full observation population and missing-label reasons. This distinction prevents a comparison from favoring a method merely because more of its unsuccessful or unobservable assignments were excluded.

Each request reserves attempts, prompt and completion tokens, tool time, and storage before dispatch. Observed costs from errors are retained. Complete cost covers producer execution, recipient integration, judgment service, communication, retries, replay, validation, and returned artifacts. We record configuration, model route/version, code/data hashes, seed, hardware, timestamps, raw responses, token use, actions, failures, unknown reasons, and per-sample results. Reports include stream-level means, medians, 95%

Cell	Track / root	Strategy / intervention	Matched comparison	Measurement / result
RQ1-I	ArtifactRole / dev	Raw vs. terminal vs. RARE	Same delivery, recipient observation, legal feedback	Brier/log loss, calibration: xx
RQ1-II	ArtifactRole / confirm	Contextual trust vs. RARE	Same read cut, delay, propensity, model, budget	Incremental prediction, transfer: xx
RQ2-I	ArtifactRole / confirm	No evidence vs. public evidence	Preview and assignment before target execution	Change, quality, adoption, rework: xx
RQ2-II	ArtifactRole / confirm	Delayed-only vs. evidence+delay	Same target and selected-only outcome	Future utility, complete cost: xx
RQ2-III	ArtifactRole / confirm	RARE vs. pooled controller	Same capacity and candidate menu	Utility, concentration, propensity: xx
RQ3-I	PeerSelect / dev stream	Uniform, no/history, updater	Pinned graph and call/token budget	Payoff, p50/p95, state bytes: xx
RQ3-II	PeerSelect / drift	Update with/without replay	Fixed peer/task drift and delay	Recovery, forgetting, backlog: xx
RQ4-I	ArtifactRole / eligible roots	Responsibility mutations	Producer, recipient, mixed, unknown, failure	Attribution, UNKNOWN, no-op: xx
RQ4-II	ArtifactRole / eligible roots	Remove one mechanism	Gate, evidence, delay, locality, updater	Utility, safety, cost, interaction: xx

Table 5: Preregistered matrix. Each cell is bound to a structural root, split, policy, seed, budget, scorer, cost record, and raw-event manifest.

Count	Definition and reporting use
$N_{\text{scheduled}}$	Preregistered, eligible targets; planned load and stopping rules.
N_{started}	Targets actually started after selection; coverage and resource use.
N_{outcome}	All started targets, including failed, unknown, and invalid outcomes; H2 ITT denominator.
N_{credit}	Subset with valid assignment, lineage, selected-only outcome, reward contract, and once-only feedback; legal updates.
N_{obs}	Complete source observations ($O_i = 1$); information coverage.
N_{attr}	Responsibility-qualified evidence ($G_i = 1$); attribution coverage.

Table 6: Outcome coverage and update eligibility are reported separately.

intervals, effect sizes, root heterogeneity, ITT and per-protocol denominators, and quality–cost Pareto points.

Development and search costs are reported separately from deployment costs. The pooled controller receives the same total capacity. Judgment-free arms either pay the declared comparison cost or share a judgment call whose output is masked. This prevents a change in information access from silently changing the charged budget. We report observation-publication, attribution-pass, and valid-update rates alongside outcomes to expose the coverage lost at each qualification step. Future assignment quality–complete-cost utility is the primary endpoint; source producer scores are auxiliary and cannot replace it.

Mechanism ablations. Count-only and judgment-masked controls preserve handoff count, arrival time, and cost while removing judgment category. The 2×2 design crosses public evaluation visibility e with validated delayed updating u. With expected utility V_{eu},

$$\begin{aligned} \Delta_{\text{obs}|\text{upd}} &= V_{11} - V_{01}, \\ \Delta_{\text{upd}|\text{obs}} &= V_{11} - V_{10}, \\ I &= V_{11} - V_{10} - V_{01} + V_{00}. \end{aligned} \tag{17}$$

These contrasts distinguish information, updating, and interaction on the same utility scale. The total difference $V_{11} - V_{00} > 0$ can arise entirely from reading evaluations and does not by itself establish an updating benefit. Masking hides evaluation from the selector, without removing audit records or skipping responsibility validation. Disabling parameter updates still allows newly visible evaluations to affect a later choice.

For example, after A evaluates B's delivery, the evaluation-only arm may choose C while keeping parameters fixed. The full arm additionally updates after a later executed task produces valid feedback. Their subsequent executed tasks, rather than only score previews, determine whether updating is useful. Neither arm may turn A's own integration work into a negative label for B.

The four arms share initial states and exogenous task schedules, but each retains its own choices, deliveries, and histories. Uncertainty is estimated over paired independent streams. Fixed-history replay tests input sensitivity, not online utility differences. Because this ablation deliberately changes visible information, it does not replace the same-information primary comparison. Both the charging rule and measured costs are reported.

Responsibility interventions fix the producer delivery and vary recipient-owned integration. We inspect attribution labels, parameter updates, and later choice probabilities separately: avoiding a false producer label does not require every choice probability to remain unchanged. Other interventions remove the responsibility gate, selected-only constraint, delay handling, locality, or replay while matching models, candidates, and budgets. Version

Endpoint	RARE	Same-info. baseline	Difference / CI
Future quality	xx	xx	xx
Complete cost	xx	xx	xx
Quality–cost utility	xx	xx	xx
Assignment change	xx	xx	xx
Judgment Brier	xx	xx	xx
Update p95 / state bytes	xx	xx	xx
Old-root forgetting	xx	xx	xx
UNKNOWN / false attribution	xx	xx	xx

Table 7: Primary results against contextual trust/bandit, the same-information baseline; intervals use independently reset streams.

tests cover duplicates, late records, and corrections across archive cuts, verifying that effective history retains only one version of each source.

7 Results

RQ1: information value. In ArtifactRole, RARE differed from raw acceptance, terminal-only feedback, and same-information contextual trust in Brier score by xx, xx, and xx (stream-level 95% CIs: xx, xx, and xx). Log loss, calibration error, and cross-root transfer were xx, xx, and xx. The evaluation included xx valid observations, an UNKNOWN rate of xx, and judgment-service cost of xx. Predictive metrics remain distinct from team utility.

RQ2: assignment change. Candidate sets, public history, propensities, and samples were sealed before target execution. We compared executed later tasks under no evidence, public evidence only, delayed updating only, and RARE. Under RARE, assignment-change rate, future quality, rework, and quality–complete-cost utility were xx, xx, xx, and xx (95% CI: xx). All scheduled targets that actually started entered the H2 ITT denominator, including quality-check, UNKNOWN, invalid, resource, and scorer failures. Only the subset passing assignment-credit validation produced L_j for delayed update and assignment credit.

RQ3: service behavior. In PeerSelect, publish, read/assignment, and update p50/p95 latency were xx, xx, and xx. Backlog, state bytes, model calls, tokens, replay time, and complete service cost were xx, xx, xx, xx, xx, and xx; the difference from matched-budget updaters was xx. This track establishes the online service boundary but does not substitute for ArtifactRole attribution evidence.

RQ4: responsibility and drift. Under recipient-only repair, mixed ownership, unknown responsibility, version replacement, out-of-order feedback, and resource failure, false producer attribution, UNKNOWN precision, observation publication, valid update, drift recovery, old-root forgetting, and no-op rate were xx, xx, xx, xx, xx, xx, and xx (95% CI: xx). Records whose responsibility could not be independently determined remained contextual observations and did not trigger producer reward or an ineligible parameter update.

Results are organized by structural root, candidate menu, public information, feedback timing, budget, and stopping rule. ITT and per-protocol denominators, costs of failed runs, unknown reasons, and version corrections are reported together. Runs with incomplete integrity records do not supply numerical effect estimates; they remain represented in coverage and missingness reports, and their incurred cost is retained. Responsibility safety, prediction accuracy, and team utility are reported separately.

Evaluation	Update	Future utility (95% CI)	Cost
Masked	Off	xx	xx
Visible	Off	xx	xx
Masked	On	xx	xx
Visible	On	xx	xx

Table 8: Evaluation–update factorial ablation.

8 Discussion and Limitations

The method applies when responsibility boundaries, artifact versions, and later outcomes can be recorded explicitly. It does not discover open-ended workflows or infer general expertise from aggregate success. PeerSelect describes responsibility opportunities within local candidate sets; broader organizational restructuring requires candidate discovery and a new cost contract.

Human-inspired collaboration does not make peer evaluation ground truth. A recipient may misjudge, report selectively, or attribute its own integration difficulty to the producer. Complete but unattributable records remain contextual observations rather than producer rewards, and missing or invalid records do not update. This protects attribution at the cost of fewer learning examples. Cross-recipient judgment, quality calibration, and adversarial-feedback modeling are directions for improving reliability without relaxing the responsibility boundary.

Waiting for an executed future assignment grounds updating in actual allocation outcomes, but delays adaptation to change. Waiting alone does not establish attribution; independent responsibility checks remain necessary. The bounded selected-only update does not solve multi-step credit assignment over long chains. Hierarchical update timescales, risk-sensitive exploration, and resource scheduling for unresolved events remain outside the present scope.

Frozen representations and a small update head reduce service cost but may under-express rapid semantic or relational drift; updating a larger encoder instead raises latency and forgetting risk. Real-time adaptation must therefore be evaluated jointly through new-task adaptation, old-task retention, and complete service cost. Parameter-efficient adaptation, compressed memory, and versioned states that support rollback offer directions for extending this trade-off.

Finally, complete lineage and audit records impose storage, privacy, and communication costs. Compressed delivery summaries reduce cost but may discard details needed for judgment; cross-tool or cross-organization deployments may not share all responsibility fields. Practical systems must trade replayability against data minimization and preserve a safe unknown branch for absent information.

9 Conclusion

Earning roles transfers a principle of human collaboration to multi-agent systems: future responsibility should depend on how partners evaluate concrete work and on whether subsequent delegation succeeds. Recipient evaluation forms task-situated role evidence; independent validation separates observation from producer attribution; and later quality and complete cost calibrate the selector. The resulting object is directly testable: whether collaborator evaluation changes future assignments and improves team performance on new tasks without sacrificing responsibility safety, service behavior, stability, or cost.

References

[1] Saaket Agashe, Yue Fan, Anthony Reyna, and Xin Eric Wang. 2025. LLM-Coordination: Evaluating and Analyzing Multi-agent Coordination Abilities in Large Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, 8053–8072. doi:10.18653/v1/2025.findings-naacl.448

[2] Elif Akata, Lion Schulz, Julian Coda-Forno, Seong Joon Oh, Matthias Bethge, and Eric Schulz. 2025. Playing Repeated Games with Large Language Models. *Nature Human Behaviour* 9, 7 (2025), 1380–1390. doi:10.1038/s41562-025-02172-y

[3] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47 (2002), 235–256. doi:10.1023/A:1013689704352

[4] Glenn W. Brier. 1950. Verification of Forecasts Expressed in Terms of Probability. *Monthly Weather Review* 78, 1 (1950), 1–3. doi:10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2

[5] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2024. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. In *International Conference on Learning Representations*. 20094–20136. https://proceedings.iclr.cc/paper_files/paper/2024/hash/578e65cdee35d00c708d4c64bce32971-Abstract-Conference.html

[6] Yuhang Chen, Yutao Sun, Hao Wang, Jiawei Wang, Xinyu Zhang, Xue Shen, Wei Li, and Weinan Zhang. 2026. Exact Is Easier: Credit Assignment for Cooperative LLM Agents. arXiv:2603.06859 https://arxiv.org/abs/2603.06859

[7] Jakob Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. 2018. Counterfactual Multi-Agent Policy Gradients. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, Vol. 32. doi:10.1609/aaai.v32i1.11794

[8] Yiming Gao, Mengdi Wang, Yaoliang Yu, Zhicheng Ma, and Anru Qu. 2026. DecisionBench: A Benchmark for Emergent Delegation in Long-Horizon Agentic Workflows. arXiv:2605.19099 https://arxiv.org/abs/2605.19099

[9] Linyan Ge and Lan-Zhe Guo. 2026. Roles with Rails: Contract-Preserving Role Evolution in Multi-Agent Structured Reasoning. arXiv:2605.28433 https://arxiv.org/abs/2605.28433

[10] Brian P. Gerkey and Maja J. Matarić. 2004. A Formal Analysis and Taxonomy of Task Allocation in Multi-Robot Systems. *The International Journal of Robotics Research* 23, 9 (2004), 939–954. doi:10.1177/0278364904045564

[11] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*. PMLR, 1321–1330. https://proceedings.mlr.press/v70/guo17a.html

[12] Zixuan Hao, Tianyi Wang, Hongwei Dong, Zihan Liu, Haoyu Wang, Xinyu Lin, Qing Lin, Chao Wang, Hao Dong, and Jie Chen. 2026. Evolve as a Team: Collaborative Self-Evolution for LLM-based Multi-Agent Systems. arXiv:2605.29790 https://arxiv.org/abs/2605.29790

[13] Pablo Hernandez-Leal, Michael Kaisers, Tim Baarslag, and Enrique Munoz de Cote. 2017. A Survey of Learning in Multiagent Environments: Dealing with Non-Stationarity. arXiv:1707.09183 https://arxiv.org/abs/1707.09183

[14] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *International Conference on Learning Representations*. Oral presentation. https://iclr.cc/virtual/2024/oral/19756

[15] Pooria Joulani, Andras Gyorgy, and Csaba Szepesvari. 2013. Online Learning under Delayed Feedback. In *Proceedings of the 30th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 28)*. PMLR, 107–115. https://proceedings.mlr.press/v28/joulani13.html

[16] Khimya Khetarpal, Matthew Riemer, Irina Rish, and Doina Precup. 2022. Towards Continual Reinforcement Learning: A Review and Perspectives. *Journal of Artificial Intelligence Research* 75 (2022), 1401–1476. doi:10.1613/jair.1.13673

[17] Yubin Kim, Chanwoo Park, Taehan Kim, Eugene Park, Samuel Schmidgall, Salman Rahman, Chunjong Park, Cynthia Breazeal, Xin Liu, Hamid Palangi, Hae Won Park, and Daniel McDuff. 2026. TeamBench: Evaluating Agent Coordination under Enforced Role Separation. *arXiv preprint arXiv:2605.07073* (2026). https://arxiv.org/abs/2605.07073

[18] Marc Lanctot, Kate Larson, Michael Kaisers, Quentin Berthet, Ian Gemp, Manfred Diaz, Roberto-Rafael Maura-Rivero, Yoram Bachrach, Anna Koop, and Doina Precup. 2025. Soft Condorcet Optimization for Ranking of General Agents. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*. IFAAMAS. https://www.ifaamas.org/Proceedings/aamas2025/pdfs/p1253.pdf

[19] Kyle Lewis. 2003. Measuring Transactive Memory Systems in the Field: Scale Development and Validation. *Journal of Applied Psychology* 88, 4 (2003), 587–604. doi:10.1037/0021-9010.88.4.587

[20] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. 2010. A Contextual-Bandit Approach to Personalized News Article Recommendation. In *Proceedings of the 19th International Conference on World Wide Web*. ACM, 661–670. doi:10.1145/1772690.1772758

[21] Peiwen Li, Shiyang Zhang, Yangtian Zhang, Sizhuang He, David van Dijk, and Rex Ying. 2026. MoRSE: Task-Oriented Multi-Agent System with Mixture of Role-Subtask Experts. arXiv:2608.09251 https://arxiv.org/abs/2608.09251

[22] Ruizhe Li, Zhen Zhang, Xinyu Bo, Qian Dai, Cheng Li, Fei Wen, and Xilin Chen. 2026. Towards Adaptive, Scalable, and Robust Coordination of LLM Agents: A Dynamic Ad-Hoc Networking Perspective. arXiv:2602.08009 https://arxiv.org/abs/2602.08009

[23] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/e9df36b21ff4ee211a8b71ee8b7e9f57-Abstract-Conference.html

[24] Thomas W. Malone and Kevin Crowston. 1994. The Interdisciplinary Study of Coordination. *Comput. Surveys* 26, 1 (1994), 87–119. doi:10.1145/174666.174668

[25] Giorgio Piatti, Zhijing Jin, Max Kleiman-Weiner, Bernhard Schölkopf, Mrinmaya Sachan, and Rada Mihalcea. 2024. Cooperate or Collapse: Emergence of Sustainable Cooperation in a Society of LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 37. 111715–111759. doi:10.52202/079017-3548

[26] Iñigo Pinyol and Jordi Sabater-Mir. 2013. Computational Trust and Reputation Models for Open Multi-Agent Systems: A Review. *Artificial Intelligence Review* 40 (2013), 607–626. doi:10.1007/s10462-011-9277-z

[27] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. 2020. Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning. *Journal of Machine Learning Research* 21, 178 (2020), 1–51. https://www.jmlr.org/papers/v21/20-081.html

[28] Siyue Ren, Wanli Fu, Xinkun Zou, Chen Shen, Yi Cai, Chen Chu, Zhen Wang, and Shuyue Hu. 2026. Reputation as a Solution to Cooperation Collapse in LLM-based MASs. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. IFAAMAS. https://www.ifaamas.org/Proceedings/aamas2026/pdfs/UEHN4980.pdf

[29] Benedict Russell, Chin wing Leung, and Paolo Turrini. 2026. Defection at First Sight: Learning Partner Selection in Optional Social Dilemmas without Prior Information. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. IFAAMAS. https://www.ifaamas.org/Proceedings/aamas2026/pdfs/IBSZ1473.pdf

[30] Jordi Sabater and Carles Sierra. 2002. Reputation and Social Network Analysis in Multi-Agent Systems. In *Proceedings of the First International Joint Conference on Autonomous Agents and Multiagent Systems*. 475–482. doi:10.1145/544741.544854

[31] W. T. Luke Teacy, Jigar Patel, Nicholas R. Jennings, and Michael Luck. 2006. TRAVOS: Trust and Reputation in the Context of Inaccurate Information Sources. *Autonomous Agents and Multi-Agent Systems* 12, 2 (2006), 183–198. doi:10.1007/s10458-006-5952-x

[32] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 16022–16076. doi:10.18653/v1/2024.acl-long.850

[33] Daniel M. Wegner. 1987. Transactive Memory: A Contemporary Analysis of the Group Mind. In *Theories of Group Behavior*. Springer New York, 185–208. doi:10.1007/978-1-4612-4634-3_9

[34] Chin wing Leung and Paolo Turrini. 2024. Learning Partner Selection Rules that Sustain Cooperation in Social Dilemmas with the Option of Opting Out. In *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*. IFAAMAS, 1110–1118. https://www.ifaamas.org/Proceedings/aamas2024/pdfs/p1110.pdf

[35] Chin wing Leung, Paolo Turrini, and Ann Nowé. 2025. Curiosity-Driven Partner Selection Accelerates Convention Emergence in Language Games. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*. IFAAMAS, 1282–1290. https://www.ifaamas.org/Proceedings/aamas2025/pdfs/p1282.pdf

[36] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *Proceedings of the First Conference on Language Modeling*. https://openreview.net/forum?id=BAakY1hNKS

[37] Yingxuan Yang, Huacan Chai, Shuai Shao, Yuanyi Song, Siyuan Qi, Renting Rui, and Weinan Zhang. 2025. AgentNet: Decentralized Evolutionary Coordination for LLM-based Multi-Agent Systems. In *Advances in Neural Information Processing Systems*, Vol. 38. https://papers.nips.cc/paper_files/paper/2025/hash/9a379c1b05793d1c42dc832269834515-Abstract-Conference.html

[38] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *International Conference on Learning Representations*. https://openreview.net/forum?id=roNSXZpUDN

[39] Mireia Yurrita, Davide Dell'Anna, Pradeep K. Murukannaiah, Catholijn M. Jonker, and Pinar Yolum. 2026. Developing Guidelines for Human-LLM Agent Teams: A Multi-Stakeholder Lens. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. IFAAMAS. doi:10.65109/JOWO4591

[40] Miao Zhang, Junsik Kim, Siyuan Xiang, Jian Gao, and Cheng Cao. 2026. Dynamic Role Assignment for Multi-Agent Debate. arXiv:2601.17152 https://arxiv.org/abs/2601.17152

[41] Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhailong Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. 2025. MultiAgentBench: Evaluating the Collaboration and Competition of LLM Agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 8580–8622. doi:10.18653/v1/2025.acl-long.421

[42] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. 2024. GPTSwarm: Language Agents as Optimizable Graphs. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 62743–62767. https://proceedings.mlr.press/v235/zhuge24a.html